Praca Inżynierska

Adrian Zygarowski

3 marca 2026

Spis treści

1 Modelowanie obiektu sterowania – Robot Follow Me **2**
1.1 Dlaczego model wejście–wyjście nie jest wystarczający dla naszego projektu? 2
1.2 Definicja zmiennych wejścia i wyjścia 2
1.3 Definicja zmiennych stanu 2
1.4 Wyprowadzenie modelu typu white-box 2

2 Schemat blokowy układu regulacji **4**

3 Architektura systemu sterowania w środowisku ROS2 **5**
3.1 Architektura pakietów, węzłów i przepływu danych 5

1 Modelowanie obiektu sterowania – Robot Follow Me

1.1 Dlaczego model wejście–wyjście nie jest wystarczający dla naszego projektu?

Nasz robot zmienia orientację ruchu w osi x, co jest zależne od kąta δ. Ruch w osi y również zależy od tego kąta. Oznacza to, że każda zmiana kąta, czyli wykonanie skrętu, powoduje zmianę przyszłego położenia robota. W konsekwencji wyjście nie jest w pełni zależne wyłącznie od bieżącego wejścia sterującego.

1.2 Definicja zmiennych wejścia i wyjścia

Wejście: prędkość liniowa robota v oraz kąt skrętu kół δ

$$u = \begin{bmatrix} v \\ \delta \end{bmatrix}$$

Wyjście: pozycja robota x, y oraz orientacja δ

$$y = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

1.3 Definicja zmiennych stanu

Definicja 1 (Zmienna stanu)**.** *Zmienna stanu to najmniejsza liczba wielkości, których znajomość wartości w chwili t_0, przy znajomości wartości wielkości wejściowych od chwili t_0, pozwala jednoznacznie określić stan układu oraz wielkości wyjściowe w dowolnej chwili przyszłej.*

Przyjmujemy następujące zmienne stanu:

$$x_1 = x, \quad x_2 = y, \quad x_3 = \delta.$$

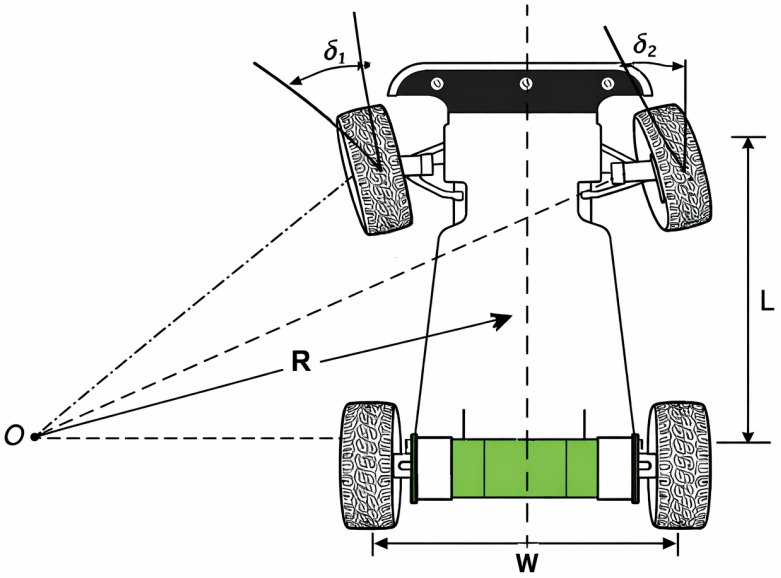
1.4 Wyprowadzenie modelu typu white-box

Definicja 2 (Model White-Box)**.** *Model typu white-box to model który w całości jest opisany z wykorzystaniem praw fizyki.*

Założenie: robot podczas ruchu nie może ślizgać się na boki, tzn. składowa prędkości prostopadła do kierunku ruchu jest równa 0.

Rozważmy model kinematyczny typu Ackermanna o rozstawie osi L.

Wyprowadzenie równań stanu



Rysunek 1: Geometria modelu Ackermanna: rozstaw osi L, szerokość toru W, promień skrętu R oraz kąty skrętu kół przednich δ_1, δ_2.

W układzie globalnym prędkość liniowa v rozkłada się na składowe (z zależności trygonometrycznych):

$$\dot{x} = v\cos(\theta) \tag{1}$$

$$\dot{y} = v\sin(\theta) \tag{2}$$

Prędkość kątowa wynosi:

$$\omega = \frac{v}{R} \quad dlatego \quad \dot{\theta} = \frac{v}{R} \tag{3}$$

Następnie musimy wyznaczyć R w zależności od kąta skrętu:

$$R = \frac{L}{\tan(\delta)} \tag{4}$$

I z powrotem podstawiamy do równania nr 3:

$$\dot{\theta} = \frac{v}{L}\tan(\delta) \tag{5}$$

Model w postaci przestrzeni stanów

Definiujemy wektor stanu:

$$x = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

oraz wektor wejścia:

$$u(t) = \begin{bmatrix} u_1(t) \\ u_2(t) \end{bmatrix} = \begin{bmatrix} v(t) \\ \theta(t) \end{bmatrix}$$

Model dynamiczny można zapisać w postaci:

$$\dot{x} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} V cos(\theta) \\ V sin(\theta) \\ \frac{v}{L}\tan(\delta) \end{bmatrix}$$

Model ten jest nieliniowy ze względu na zawieranie funkcji trygonometrycznych zmiennej stanu θ oraz wejścia δ.

Wyprowadzenie równań wyjść

Równania wyjścia odpowiadają bezpośrednio zmiennym stanu:

$$y_1(t) = x(t) \tag{6}$$

$$y_2(t) = y(t) \tag{7}$$

$$y_3(t) = \theta(t) \tag{8}$$

Zapis wektorowo-macierzowy:

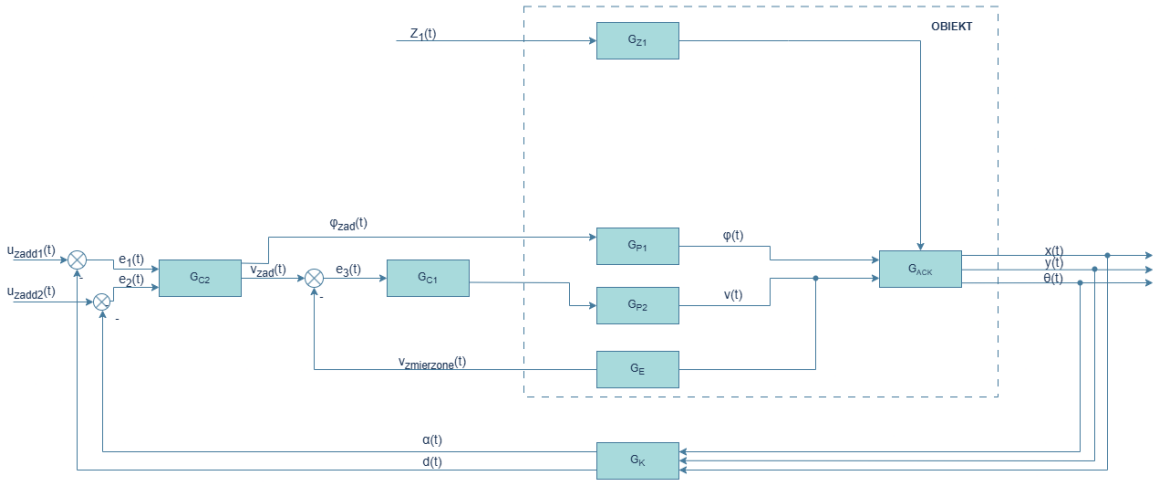
$$\mathbf{y}(t) = \begin{bmatrix} y_1(t) \\ y_2(t) \\ y_3(t) \end{bmatrix}$$

I ponieważ wyjście jest bezpośrednio równe zmiennym stanu, macierz wyjścia C jest macierzą jednostkową:

$$\mathbf{y}(t) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x(t) \\ y(t) \\ \theta(t) \end{bmatrix}$$

2 Schemat blokowy układu regulacji

Celem pracy jest zaprojektowanie układu sterowania umożliwiającego autonomiczne podążanie robota za wykrytym obiektem przy jednoczesnym utrzymywaniu zadanej odległości pomiędzy robotem a obiektem.



Rysunek 2: Schemat blokowy kaskadowego układu regulacji robota mobilnego

Na rysunku 2 przedstawiono schemat blokowy zaprojektowanego kaskadowego układu regulacji.

- u_{zad1} – wartość zadana podawana przez użytkownika określająca odległość robota od śledzonego obiektu.

- u_{zad2} – wartość zadana kąta równa 0°, oznacza domyslnie ruch robota na wprost..
- e_1 – uchyb odległości, będący różnicą pomiędzy zadaną odległością a rzeczywistą odległością robota od obiektu.
- e_2 – uchyb kierunku, określający różnicę pomiędzy kątem z którym porusza się robota a kierunkiem położenia obiektu względem osi podłużnej robota.
- G_{C2} – regulator nadrzędny. Na podstawie uchybów e_1 oraz e_2 wyznacza zadane wartości prędkości liniowej robota oraz zadanego kąta skrętu kół.
- G_{C1} – regulator podrzędny (regulator prędkości). Jego zadaniem jest utrzymywanie rzeczywistej prędkości robota na poziomie zadanym przez regulator nadrzędny.
- e_3 – uchyb prędkości, stanowiący różnicę pomiędzy prędkością zadaną a prędkością zmierzoną za pomocą enkoderów.
- G_{P1} – serwomechanizm odpowiedzialny za realizację zadanego kąta skrętu kół przednich.
- G_{P2} – układ napędowy robota, obejmujący silnik prądu stałego wraz ze sterownikiem.
- z_1 – zakłócenie działające na ruch robota.
- G_{Ack} – model kinematyczny wyprowadzonego pzez nas robota typu Ackermanna..
- G_K – tor pomiarowy wizyjny (kamera). Na podstawie obrazu wyznacza odległość robota od obiektu oraz kąt położenia obiektu względem osi robota, które następnie wykorzystywane są do wyznaczenia uchybów regulacji.

3 Architektura systemu sterowania w środowisku ROS2

Pakiety zostały podzielone ze względu na funkcje, jakie pełnią poszczególne elementy w układzie regulacji.
TUTAJ BĘDZIE GRAF Z PAKIETAMI/TEMATAMI/WĘZŁAMI POD KONIEC

3.1 Architektura pakietów, węzłów i przepływu danych

1. **Pakiet robot_calibration**:

   `Initialization_node` publikuje na temat $/Target_config$, subskrybowany przez `Follow_Me_node` oraz `Target_Detection_node`.

2. **Pakiet robot_perception**:

   `Target_Detection_node` publikuje na temat $/Target_Detection$ oraz przekazuje dane do `Estimate_Target_node`.

   `Estimate_Target_node` publikuje na temat $/Target_Pose$, który jest subskrybowany przez `Follow_Me_node`.

3. **Pakiet robot_control**:

   `Follow_Me_node` publikuje na temat $/cmd_ackermann$, subskrybowany przez `PiRacer_Control_node` oraz `odometry_node`.

4. **Pakiet piracer_control**:

   `PiRacer_Control_node` publikuje na temat $/cmd_motion$, subskrybowany przez `PWM_Driver_node`,

   oraz publikuje na temat $/Steering_Servo$, subskrybowany przez `Servo_Driver_node`.

   `odometry_node` publikuje na temat $/odom$, subskrybowany przez `Follow_Me_node`.

5. **Pakiet robot_hardware_interface**:

   `PWM_Driver_node`, `Servo_Driver_node`,

   `Encoder_Reader_node` publikuje na temat $/Wheel_speed$, subskrybowany przez `PiRacer_Control_node` oraz `odometry_node`.

Odometria jest niezbędna do określenia pozycji robota w przestrzeni. Wykorzystuje ona dane publikowane w tematach $/wheel_speed$ oraz $/cmd_ackermann$. Węzeł `odometry_node` oblicza prędkość środka osi tylnych kół zgodnie z zależnością $V = \omega R$.

Kąt skrętu δ pobierany jest z tematu $/cmd_ackermann$, natomiast rozstaw osi L z modelu URDF. Kluczowym parametrem w procesie całkowania jest czas pomiędzy kolejnymi iteracjami algorytmu.

$$\dot{\theta} = \frac{v}{L}\tan(\delta) \tag{9}$$

Po scałkowaniu względem czasu otrzymujemy:

$$\theta(t) = \theta_0 + \int_{t_0}^{t} \frac{v(\tau)}{L}\tan\big(\delta(\tau)\big)\,d\tau \tag{10}$$

Teraz należy zdyskretyzować powyższe rownanie z wykorzystaniem metody rapezów.
Aproksymacja całki metodą trapezów ma postać:

$$y(kT_s) = \frac{T_s}{2}\sum_{j=0}^{k-1}\big(u(j+1) + u(j)\big) \tag{11}$$

Równoważna postać rekurencyjna wyraża się wzorem:

$$y(k) = y(k-1) + \frac{T_s}{2}\big(u(k) + u(k-1)\big) \tag{12}$$

Podstawiając do naszego wcześniejszego równania

$$\dot{\theta} = \frac{v}{L}\tan(\delta), \tag{13}$$

otrzymujemy dyskretną postać równania orientacji:

$$\theta(k) = \theta(k-1) + \frac{T_s}{2}\left(\frac{v(k)}{L}\tan\big(\delta(k)\big) + \frac{v(k-1)}{L}\tan\big(\delta(k-1)\big)\right). \tag{14}$$

Czyli kluczowe jest uzyskanie dt=const pomoze nam to uzyskac wbudowany zegar ROS wykorzystac komende now=self.get$_c lock().now()tozwrocinamczasROSwceluuzysjaniaczasudt = (now - self.lasst_t ime).nanoseconds$
$1e-9$.